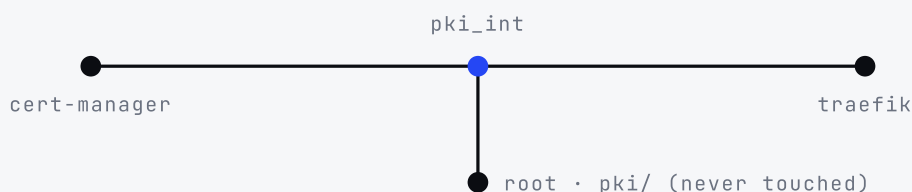


INTERNAL PKI WITH OPENBAO

An internal CA whose key never leaves the vault: root and intermediate in OpenBao, certificates via cert-manager, renewal and revocation.



PKI with OpenBao and cert-manager

An internal CA whose key never leaves the vault: root and intermediate in OpenBao, certificates via cert-manager for ingress and service-to-service, renewal and revocation

Thomas Zachmann

17 September 2026

Contents

What this is about

- Why by hand, and why without AI

- License and liability

- Who this is for

- The building blocks

- The architecture in one picture

Part I – The starting point

Part II – Root and intermediate in OpenBao

- Root CA

- Intermediate CA

- The role: what gets signed

- Policy and auth role for cert-manager

Part III – Connecting cert-manager

- ServiceAccount, RBAC, ClusterIssuer

- The first certificate

- Checking the chain

Part IV – What gets rejected

Part V – Ingress with ingress-shim

Part VI – Renewal and revocation

- Renewal

- Revocation

Part VII – Operation

- Distributing the root

- Retiring the old CA

- Root rotation, in five years

- What belongs in OpenTofu

- What is still open

Appendix A – All commands

Appendix B – Glossary

What this is about

Every cluster needs its own CA at some point. For ingress hosts that are not public, for TLS between services, for webhooks, for everything that wants a certificate and has no Let's Encrypt route. The usual way is cert-manager with a self-signed root: two manifests, five minutes, done – and the CA's private key sits as a Kubernetes Secret in the cert-manager namespace. Anyone with `get secrets` there can issue certificates for any hostname in the cluster, and nobody finds out.

This guide moves the CA into OpenBao. Root and intermediate CA are created there; the key never leaves OpenBao. cert-manager still generates the certificates' private keys inside the cluster, but only sends the CSR to OpenBao – through a role that defines which hostnames, which key types and which lifetimes are allowed. Every issuance is traceable in OpenBao, every revocation lands on the CRL.

It is the fifth part of a series about OpenBao in a homelab. N° 1 builds the OpenBao in the cluster, N° 2 and N° 4 let the External Secrets Operator draw secrets from it. This guide can be read on its own but assumes the OpenBao from N° 1 with Kubernetes auth enabled.

Everything was carried out on the cluster. The two errors that came up are in it – one of them was not the expected one.

Why by hand, and why without AI

A CA hierarchy is described in twenty lines of HCL, and a language model writes them without error. What it does not deliver: why the intermediate CSR leaves OpenBao but the root never does; why set-signed suddenly creates two issuers; why a certificate with an empty `subject=` is still valid; and why the first rejection was not about the hostname but about the key type.

Tools like [nyrvex](#), which generates the configuration of an AI platform's secret store and identity provider, take these steps off your hands later. You should have walked them yourself once, to be able to judge what was generated.

The yardstick: whoever has worked through this guide can draw, on a blank sheet, which key is created where, which artefact travels between cluster and OpenBao, and at which point a role decides whether a certificate is issued.

License and liability

This guide is licensed under CC BY 4.0: it may be copied, shared and adapted, commercially too, as long as the author is credited. It is provided as is, without warranty. Everything in it was carried out in a development environment; whoever reproduces it elsewhere does so at their own risk.

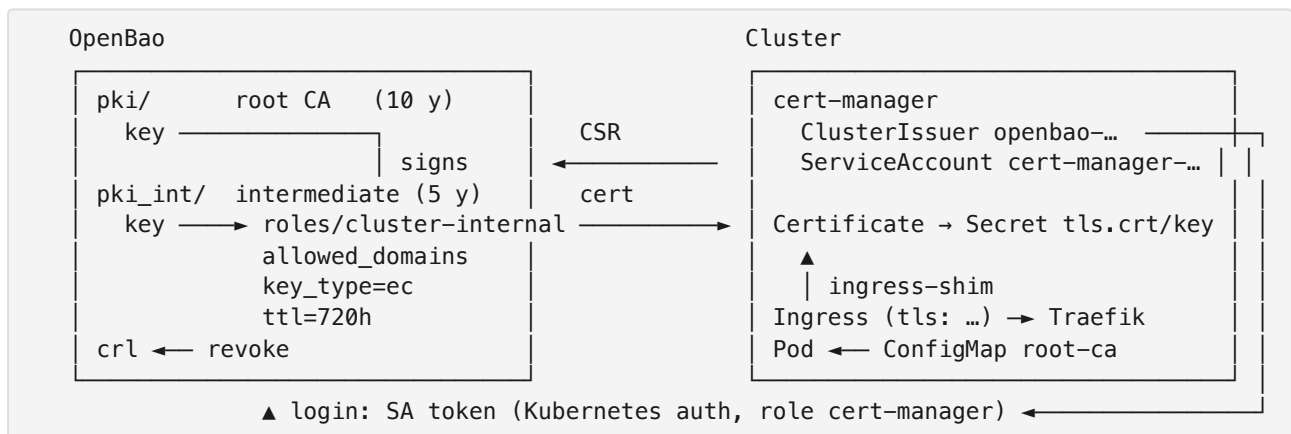
Who this is for

Readers who have used cert-manager before and know what a CSR is. OpenBao basics – mounts, roles, policies, Kubernetes auth – are assumed; N° 1 of this series or my book **Vault in Practice** (Leanpub, leanpub.com/vault-in-practice) provide them.

The building blocks

Component	Version	Role
OpenBao in the cluster (N° 1)	2.6.2	PKI engines pki/ and pki_int/, Kubernetes auth
cert-manager	v1.20.2	issuer of type vault, Certificate, ingress-shim
Traefik	RKE2 chart 40.1	ingress with TLS from the cert-manager Secret
Miniflux (N° 2)	2.3.3	the service that gets a certificate

The architecture in one picture



Three things to take from the picture:

1. **Two keys, two places.** The CA key is created in OpenBao and stays there. The certificate key is created in the cluster (cert-manager) and stays there. Only the CSR and the signed certificate travel.
2. **The role is the PKI's policy.** What OpenBao signs is decided not by cert-manager but by **pki_int/roles/ccluster-internal**: allowed domains, key type, lifetime. cert-manager can only apply.
3. **Root and intermediate are separate mounts.** The root signs exactly once – the intermediate – and is not touched afterwards. If the intermediate is compromised, you revoke it and issue a new one; the root stays trustworthy.

Part I – The starting point

This is what the cluster looked like before this guide:

```
$ kubectl get clusterissuer
NAME                READY   AGE
homelab-ca          True    3d22h
selfsigned-bootstrap True    3d22h

$ kubectl get clusterissuer homelab-ca -o jsonpath='{.spec}'
{"ca":{"secretName":"homelab-ca-key-pair"}}

$ kubectl get certificate -n cert-manager homelab-ca -o jsonpath='{.spec.isCA} {.spec.duration}'
true 87600h
```

A self-signed root, valid for ten years, key in `cert-manager/homelab-ca-key-pair`. Two services draw certificates from it. It works, and it is the state most clusters are in.

What is missing: the key sits in `etcd`, readable by anyone with `Secret` rights in the namespace. There is no intermediate – every certificate hangs directly off the root. There is no record of who issued which certificate when, apart from the `CertificateRequest` objects, which get cleaned up eventually. And there is no revocation: `cert-manager` knows no CRL.

This guide leaves the old CA in place – migrating running services is a topic of its own (Part VII) – and builds the new one next to it.

Part II – Root and intermediate in OpenBao

Root CA

```
/ $ bao secrets enable -path=pki -max-lease-ttl=87600h pki
/ $ bao write pki/root/generate/internal \
    common_name="Homelab Root CA" issuer_name=root-2026 \
    key_type=ec key_bits=256 ttl=87600h
```

generate/internal means: the key is generated in OpenBao and **not returned**. The response contains the certificate, the issuer ID and the key ID, but no private key. (generate/exported would return it – for backup scenarios you then really need.)

EC P-256 instead of RSA: smaller keys, faster signatures, supported by everything modern. ttl=87600h is ten years – the mount's max-lease-ttl must be at least that, otherwise OpenBao caps silently.

```
/ $ bao write pki/config/urls \
    issuing_certificates="http://openbao.openbao.svc:8200/v1/pki/ca" \
    crl_distribution_points="http://openbao.openbao.svc:8200/v1/pki/crl"
```

The URLs end up as AIA and CRL distribution point in every issued certificate. Without them OpenBao warns on every signing, and clients that want to check revocations cannot find the CRL.

Intermediate CA

A second mount, its own key, half the lifetime:

```
/ $ bao secrets enable -path=pki_int -max-lease-ttl=43800h pki
/ $ bao write -field=csr pki_int/intermediate/generate/internal \
    common_name="Homelab Intermediate CA" key_type=ec key_bits=256 > /tmp/pki_int.csr
```

This is the only moment anything comes out of the intermediate mount – a **CSR**, not a key. It goes to the root:

```
/ $ bao write -field=certificate pki/root/sign-intermediate \
    csr=@/tmp/pki_int.csr format=pem_bundle ttl=43800h issuer_ref=root-2026 > /tmp/pki_int.pem
/ $ bao write pki_int/intermediate/set-signed certificate=@/tmp/pki_int.pem
mapping    map[0f271d38-...: 84eb727c-...:31c9adfe-...]
```

The mapping reveals something you need to know: set-signed created **two** issuers. 84eb727c is the intermediate with key 31c9adfe; 0f271d38 is a copy of the root certificate **without** a key, imported so that the mount can serve the complete chain. The issuer_name given at generate got lost along the way:

```
/ $ bao read -field=issuer_name pki_int/issuer/84eb727c-...
(empty)
/ $ bao write pki_int/issuer/84eb727c-... issuer_name=int-2026
/ $ bao write pki_int/issuer/0f271d38-... issuer_name=root-2026-chain
```

Names are not just cosmetics: `issuer_ref=int-2026` in a role survives a rotation, a UUID does not. The mount's default issuer is already the right one (`bao read pki_int/config/issuers`).

```
/ $ bao write pki_int/config/urls \
    issuing_certificates="http://openbao.openbao.svc:8200/v1/pki_int/ca" \
    crl_distribution_points="http://openbao.openbao.svc:8200/v1/pki_int/crl"
/ $ bao read -field=certificate pki_int/cert/ca | openssl x509 -noout -subject -issuer -dates
subject=CN=Homelab Intermediate CA
issuer=CN=Homelab Root CA
notBefore=Sep 17 11:13:42 2026 GMT
notAfter=Sep 16 11:14:12 2031 GMT
```

The role: what gets signed

```
/ $ bao write pki_int/roles/cluster-internal \
    allowed_domains="svc.cluster.local,demo.svc,example.internal" \
    allow_subdomains=true allow_bare_domains=false allow_glob_domains=false \
    allow_ip_sans=false enforce_hostnames=true \
    key_type=ec key_bits=256 \
    ttl=720h max_ttl=2160h \
    server_flag=true client_flag=false require_cn=false
```

Every field is a decision:

Field	Value	Why
<code>allowed_domains + allow_subdomains</code>	cluster DNS and the internal zone	everything else is rejected – including <code>example.com</code>
<code>allow_bare_domains=false</code>	–	<code>svc.cluster.local</code> itself gets no certificate, only subdomains
<code>allow_ip_sans=false</code>	–	no IP certificates; services have names
<code>key_type=ec</code>	P-256	the role enforces the type, cert-manager has to deliver it (Part IV)
<code>ttl=720h, max_ttl=2160h</code>	30 / 90 days	short enough that renewal gets exercised
<code>server_flag, client_flag</code>	server only	a certificate for service-to-service client auth would need a second role
<code>require_cn=false</code>	–	modern clients check SANs, not CN; cert-manager often sends none

Policy and auth role for cert-manager

```
/ $ bao policy write cert-manager - <<'EOF'
path "pki_int/sign/cluster-internal" { capabilities = ["create", "update"] }
EOF
/ $ bao write auth/kubernetes/role/cert-manager \
    bound_service_account_names=cert-manager-openbao \
    bound_service_account_namespaces=cert-manager \
    token_policies=cert-manager token_ttl=10m
```

One line of policy. cert-manager may call exactly one path: sign with this role. Not issue (which would have OpenBao generate the key), not pki_int/*, nothing on the root.

The complete OpenBao setup is in `openbao/pki-setup.sh`.

Part III – Connecting cert-manager

ServiceAccount, RBAC, ClusterIssuer

cert-manager does not log in to OpenBao with its own identity but with the token of a ServiceAccount created for that purpose. For that it needs the right to mint tokens for this account:

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: cert-manager-openbao
  namespace: cert-manager
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: cert-manager-openbao-token
  namespace: cert-manager
rules:
- apiGroups: [""]
  resources: ["serviceaccounts/token"]
  resourceNames: ["cert-manager-openbao"]
  verbs: ["create"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: cert-manager-openbao-token
  namespace: cert-manager
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: cert-manager-openbao-token
subjects:
- kind: ServiceAccount
  name: cert-manager
  namespace: cert-manager
---
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: openbao-cluster-internal
spec:
  vault:
    server: http://openbao.openbao.svc:8200
    path: pki_int/sign/cluster-internal
    auth:
      kubernetes:
        mountPath: /v1/auth/kubernetes
        role: cert-manager
        serviceAccountRef:
          name: cert-manager-openbao
```

The ServiceAccount lives in the cert-manager namespace because a ClusterIssuer resolves its serviceAccountRef there. With a namespace-bound Issuer it would live in the issuer's namespace – then you can give each namespace its own OpenBao role and its own allowed_domains. For a homelab cluster the ClusterIssuer is enough.

```
$ kubectl apply -f k8s/clusterissuer.yaml
$ kubectl get clusterissuer openbao-cluster-internal
NAME                                READY   AGE
openbao-cluster-internal            True    8s
$ kubectl get clusterissuer openbao-cluster-internal -o jsonpath='{.status.conditions[0].message}'
Vault verified
```

Ready on the first try. The issuer type is called vault because OpenBao kept the Vault API; cert-manager sees no difference.

The first certificate

```
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: miniflux-internal
  namespace: demo
spec:
  secretName: miniflux-internal-tls
  duration: 720h
  renewBefore: 240h
  privateKey:
    algorithm: ECDSA
    size: 256
    rotationPolicy: Always
  dnsNames:
  - miniflux.demo.svc
  - miniflux.demo.svc.cluster.local
  issuerRef:
    kind: ClusterIssuer
    name: openbao-cluster-internal
```

```
$ kubectl apply -f k8s/miniflux-certificate.yaml
$ kubectl get certificate,certificaterequest -n demo
```

NAME	READY	SECRET	AGE
certificate.cert-manager.io/miniflux-internal	True	miniflux-internal-tls	9s

NAME	APPROVED	READY	ISSUER
certificaterequest.cert-manager.io/miniflux-internal-1r-internal	True	True	openbao-cluste

What OpenBao issued:

```
$ kubectl get secret -n demo miniflux-internal-tls -o jsonpath='{.data.tls\.crt}' \
| base64 -d | openssl x509 -noout -subject -issuer -dates -ext subjectAltName
subject=
issuer=CN=Homelab Intermediate CA
notBefore=Sep 17 11:15:03 2026 GMT
notAfter=Oct 17 11:15:33 2026 GMT
X509v3 Subject Alternative Name: critical
DNS:miniflux.demo.svc, DNS:miniflux.demo.svc.cluster.local
```

subject= is empty – no CN. That is correct and has been standard for years: browsers and TLS libraries check the SANs. `require_cn=false` in the role allows it, and cert-manager sends none without `commonName` in the manifest.

`rotationPolicy`: Always makes sure a new private key is created on every renewal. The default Never keeps the key across renewals – convenient, but a compromised key then stays valid as long as the Certificate object exists.

Checking the chain

The Secret contains three things:

```
$ kubectl get secret -n demo miniflux-internal-tls -o jsonpath='{.data}' | ...
ca.crt    615 bytes, 1 cert    ← the root
tls.crt   1689 bytes, 2 certs  ← leaf + intermediate
tls.key   227 bytes
```

`tls.crt` is the **chain** a server serves: its own certificate first, then the intermediate. `ca.crt` is the root a client has to trust. The manual check, with the root straight from OpenBao:

```
$ bao read -field=certificate pki/cert/ca > root.pem
$ openssl verify -CAfile root.pem -untrusted intermediate.pem leaf.pem
leaf.pem: OK
```

`-untrusted` is the right place for the intermediate: it is used to build the chain but not trusted per se – only the root from `-CAfile` is.

Part IV – What gets rejected

A certificate for `miniflux.example.com` – a domain not in `allowed_domains`. Expected: rejection because of the hostname. Received:

```
$ kubectl get certificaterequest -n demo not-allowed-1 -o jsonpath='{.status.conditions[?(@.type=="Ready")].message}'
Vault failed to sign certificate: failed to sign certificate by vault: Error making API request.

URL: POST http://openbao.openbao.svc:8200/v1/pki_int/sign/cluster-internal
Code: 400. Errors:

* role requires keys of type ec
```

The test manifest had no `privateKey` block. `cert-manager` then uses **RSA 2048** – and the role demands EC. OpenBao checks the key type **before** the hostnames. Two consequences: every Certificate against this issuer needs `privateKey.algorithm: ECDSA`, and with ingress annotations (Part V) the algorithm has to be set as well. Or you set the role to `key_type=any` – then the applicant decides, which is defensible in a homelab PKI, less so in a corporate one.

With ECDSA in the manifest the expected error arrives:

```
Code: 400. Errors:

* subject alternate name miniflux.example.com not allowed by this role
```

Both are rejections by OpenBao, not by `cert-manager`. The `CertificateRequest` stays at `READY` False, `cert-manager` retries with backoff, and the message is in the conditions. That is exactly how a PKI should behave: the role decides, the client gets a clear answer.

Part V – Ingress with ingress-shim

For ingress hosts you do not have to write a Certificate. One annotation is enough; cert-manager creates the object from `spec.tls`:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: miniflux
  namespace: demo
  annotations:
    cert-manager.io/cluster-issuer: openbao-cluster-internal
    cert-manager.io/private-key-algorithm: ECDSA
spec:
  ingressClassName: traefik
  tls:
    - hosts:
        - miniflux.demo.example.internal
      secretName: miniflux-ingress-tls
  rules:
    - host: miniflux.demo.example.internal
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: miniflux
                port:
                  number: 8080
```

The second annotation is the lesson from Part IV. Without it: RSA, rejection, no TLS at the ingress.

```
$ kubectl apply -f k8s/miniflux-service-ingress.yaml
$ kubectl get certificate -n demo
NAME                                READY    SECRET                                AGE
miniflux-ingress-tls               True     miniflux-ingress-tls                 10s
miniflux-internal                   True     miniflux-internal-tls                 90s
```

Checking against Traefik – via SNI, because the hostname has no DNS entry:

```
$ openssl s_client -connect <worker-ip>:443 -servername miniflux.demo.example.internal -CA
file root.pem
issuer=CN=Homelab Intermediate CA
Verification: OK
Verify return code: 0 (ok)

$ curl --cacert root.pem --resolve miniflux.demo.example.internal:443:<worker-ip> \
  https://miniflux.demo.example.internal/ -o /dev/null -w '%{http_code}\n'
200
```

Traefik serves leaf and intermediate from the Secret, the client has the root, the chain closes. This is the same kind of hostname for which the reverse proxy in front of the cluster needed a public certificate until now – for purely internal services the own CA is enough from here on.

Part VI – Renewal and revocation

Renewal

cert-manager renews automatically once `renewBefore` is reached – here ten days before expiry. It can be forced by deleting the Secret:

```
$ kubectl delete secret -n demo miniflux-internal-tls
$ kubectl get certificate -n demo miniflux-internal -o jsonpath='{.status.revision}'
2
$ kubectl get certificaterequest -n demo | grep miniflux-internal
miniflux-internal-2    True    True    openbao-cluster-internal
```

New serial number, new key (`rotationPolicy: Always`), new Secret. A pod that mounts the Secret as a file sees the new file after a short while; one that loads it into memory at startup needs a restart – N° 4 describes Reloader for that.

Revocation

The old certificate is still valid until October. cert-manager no longer knows about it; OpenBao does:

```
/ $ bao list pki_int/certs
3 entries

/ $ bao write pki_int/revoke serial_number=2e:5b:a9:60:...
revocation_time_rfc3339    2026-09-17T11:17:35Z
state                      revoked

/ $ bao read -field=certificate pki_int/cert/crl | openssl crl -noout -text | grep -A2 'Revoked'
Revoked Certificates:
    Serial Number: 2E5BA9605CB696BD585DA9E766D0AF337302027B
    Revocation Date: Sep 17 11:17:35 2026 GMT
```

The revocation is on the CRL, available under the URL from `config/urls`. Whether a client checks it is its business – most internal services do not. The value of revocation therefore lies less in the technical effect than in **traceability**: OpenBao knows every issued certificate, its serial number and its status. With an audit device enabled (N° 1, Part VI), also who requested it.

With short lifetimes – 30 days here – expiry is the more effective block anyway. That is the reason to keep the role's `ttl` small.

Part VII – Operation

Distributing the root

For pods to trust the new CA they need the root certificate – only the root, not the intermediate, which comes with every chain:

```
$ bao read -field=certificate pki/cert/ca > root.pem
$ kubectl create configmap -n demo homelab-root-ca --from-file=ca.crt=root.pem
```

One ConfigMap per namespace is fine for two namespaces. Beyond that, **trust-manager** (from the cert-manager project) is the tool: a Bundle object that syncs a CA into every namespace. It was not installed here because the cluster does not need it yet.

Retiring the old CA

homelab-ca keeps issuing certificates. The migration per service: change issuerRef in the Certificate (or the ingress annotation) to openbao-cluster-internal, add privateKey.algorithm: ECDSA, delete the Secret, wait. The service then has a certificate from the new CA; its clients have to know the new root. During the transition, clients trust both roots – which is why a root change never happens in one day.

When no service depends on homelab-ca any more: delete the ClusterIssuer, delete the Secret homelab-ca-key-pair. From then on there is no CA key in etcd.

Root rotation, in five years

The intermediate expires in 2031, the root in 2036. OpenBao's issuer model is built for that: a new intermediate is created as a second issuer in the same mount, the role points to the new one via issuer_ref, the old one stays for chain building of existing certificates until the last one has expired. Hence the names (int-2026) instead of UUIDs.

What belongs in OpenTofu

Everything from openbao/pki-setup.sh – with one caveat: generate/internal for the root is a one-time act. vault_pki_secret_backend_root_cert can model it, but an accidental tofu destroy would delete the root. Mounts, roles, policies and the auth role belong in Tofu; the CA creation itself belongs in a runbook with prevent_destroy.

What is still open

- No audit device (N° 1, Part V). Until it is enabled, OpenBao knows *what* was issued but not *who* requested it.

- Server certificates only. mTLS between services needs a second role with `client_flag=true` and a way to distribute client certificates.
- No OCSP. OpenBao can do it (`ocsp_servers` in `config/urls`); Traefik does not query it.

Appendix A – All commands

```
# — OpenBao (see openbao/pki-setup.sh) —————
bao secrets enable -path=pki -max-lease-ttl=87600h pki
bao write pki/root/generate/internal common_name="Homelab Root CA" issuer_name=root-2026 key_type=ec key_bits=256 ttl=87600h
bao write pki/config/urls issuing_certificates=.../v1/pki/ca crl_distribution_points=.../v1/pki/crl
bao secrets enable -path=pki_int -max-lease-ttl=43800h pki
bao write -field=csr pki_int/intermediate/generate/internal common_name="Homelab Intermediate CA" key_type=ec key_bits=256 > pki_int.csr
bao write -field=certificate pki/root/sign-intermediate csr=@pki_int.csr format=pem_bundle ttl=43800h issuer_ref=root-2026 > pki_int.pem
bao write pki_int/intermediate/set-signed certificate=@pki_int.pem
bao write pki_int/issuer/<id> issuer_name=int-2026
bao write pki_int/config/urls issuing_certificates=.../v1/pki_int/ca crl_distribution_points=.../v1/pki_int/crl
bao write pki_int/roles/cluster-internal allowed_domains=... allow_subdomains=true key_type=ec ttl=720h max_ttl=2160h ...
bao policy write cert-manager - <<'EOF'
path "pki_int/sign/cluster-internal" { capabilities = ["create", "update"] }
EOF
bao write auth/kubernetes/role/cert-manager bound_service_account_names=cert-manager-openbao \
    bound_service_account_namespaces=cert-manager token_policies=cert-manager token_ttl=10m

# — cert-manager —————
kubectl apply -f k8s/clusterissuer.yaml
kubectl apply -f k8s/miniflux-certificate.yaml
kubectl apply -f k8s/miniflux-service-ingress.yaml
kubectl get clusterissuer,certificate,certificaterequest -A

# — verify —————
bao read -field=certificate pki/cert/ca > root.pem
openssl verify -CAfile root.pem -untrusted intermediate.pem leaf.pem
openssl s_client -connect <worker>:443 -servername miniflux.demo.example.internal -CAfile root.pem

# — renew / revoke —————
kubectl delete secret -n demo miniflux-internal-tls # forces reissue
bao list pki_int/certs
bao write pki_int/revoke serial_number=<serial>
bao read -field=certificate pki_int/cert/crl | openssl crl -noout -text
```

Appendix B – Glossary

Root CA / intermediate CA – The root signs only the intermediate; the intermediate signs everything else. Compromise of the intermediate costs a new intermediate, not a new root.

CSR – Certificate Signing Request: public key plus requested names, signed by the owner of the private key. The only thing cert-manager sends to OpenBao.

Issuer (OpenBao) – A CA certificate with (optional) key in a PKI mount. A mount can hold several; `issuer_ref` selects.

Role (PKI) – Rules for certificates: domains, key type, lifetime, usage. `sign/<role>` signs a CSR according to these rules.

sign vs. issue – `sign` takes a CSR and returns a certificate; `issue` also generates the key in OpenBao and hands it out. cert-manager uses `sign`.

ClusterIssuer / Issuer – cert-manager object describing where certificates come from. Cluster-wide or per namespace.

ingress-shim – The part of cert-manager that creates a Certificate from an Ingress's `spec.tls` and the `cert-manager.io/cluster-issuer` annotation.

SAN – Subject Alternative Name. The names a certificate is valid for; the CN is decoration by now.

AIA / CRL DP – URLs in the certificate where clients find the issuer certificate and the revocation list.

rotationPolicy – Whether cert-manager generates a new private key on renewal (Always) or keeps the old one (Never).

The CA's key sits as a Kubernetes Secret in etcd. Anyone with get secrets can issue certificates. Nobody finds out.

This guide moves the CA into OpenBao: root and intermediate as separate mounts, a role that dictates domains, key type and lifetime, and cert-manager that only sends CSRs. Ingress by annotation, service-to-service by Certificate, chain verified against Traefik.

With two rejections, the first of which was not the expected one: role requires keys of type ec came before not allowed by this role. Plus renewal with a new key, revocation on the CRL, and why set-signed creates two issuers.

```
-- The only thing that ever leaves OpenBao: a CSR
bao write -field=csr pki_int/intermediate/generate/internal ...
bao write pki/root/sign-intermediate csr=@pki_int.csr ...
bao write pki_int/intermediate/set-signed certificate=@pki_int.pem
```



openbao	2.6.2 · pki/ + pki_int/
cert-manager	v1.20.2 · issuer vault
traefik	RKE2 chart 40.1 · SNI
keys	EC P-256 · 10y / 5y / 30d
policy	1 line · sign only

20 pages · 7 parts · 2 appendices
Deep dive: **Vault in Practice**
leanpub.com/vault-in-practice
Set in Inter Tight, Inter and JetBrains Mono.
All output is real terminal output.
CC BY 4.0 · share freely · no warranty